Übungsblatt 4: Gemeinsame Ressourcen, Deadlock und Prioritätsinversion

Musterlösung · Echtzeitsysteme (EZS)

Prof. Dr.-Ing. Daniel Klünder

Musterlösung

Ziel und Anschluss an Vorlesung 3

Sie bestimmen den nötigen Schutz gemeinsamer Daten, unterscheiden Mutexbesitz und Semaphor-Token und begründen die Wartezeiten einer Ressourcenabhängigkeit. Ein Thread ist ein fortsetzbarer Ablauf; FreeRTOS nennt ihn Task. Ready heißt ausführungsbereit, Running zugeteilt und Blocked wartend.

Teil A ist ohne Hardware vollständig lösbar und klausurrelevant. Teil B verwendet den funktionsfähigen **NUCLEO-H755ZI-Q / STM32H755ZIT6**-Grundstand aus Vorlesung 3: ein FreeRTOS-Kernel auf dem Cortex-M4 über **CMSIS-RTOS2**, die Betriebssystem-Schnittstelle des Common Microcontroller Software Interface Standard. Cortex-M7 behält seinen dokumentierten System- und Startanteil. Programmiert wird mit **STM32CubeMX und VS Code mit STM32 VS Code Extension**, der Build verwendet die erzeugten **CMake-Ziele**.

Für vier Schulstunden ist die Übung in die Erklärblöcke eingebettet: zuerst 20 Minuten für Aufgabe 1 und Teil 2.1, nach der Semaphor-/Deadlock-Erklärung 15 Minuten für Teile 2.2–2.3 und Aufgabe 3, nach dem Inversionsmodell 15 Minuten für Aufgabe 4. Danach folgen 10 Minuten Codeintegration und 25 Minuten Variantenvergleich. Zusammen mit 75 Minuten Erklärung und 20 Minuten Quiz/Abschluss ergibt das 180 Minuten. Werkzeuge und Dual-Core-Grundstart müssen vor dem Termin vorbereitet sein. Eine frische Installation und erstmalige Bootfehlersuche ist keine realistische Zusatzaufgabe in diesem Zeitfenster. Bei Demonstration statt eigener Ausführung wird diese Nachweisgrenze dokumentiert.

Teil A: Papieraufgaben

Aufgabe 1: Verlorenes Update und konsistenter Snapshot (6 Punkte)

Maschinenmodell, kein nachgewiesenes Compilerlisting: Gemeinsamer Zähler startet bei 5. Jede Erhöhung besteht aus Laden, Addieren und Schreiben. A liest 5 und berechnet im eigenen Register 6. Danach wird A unterbrochen. B führt sein vollständiges Inkrement aus. Anschließend schreibt A seinen gesicherten Registerwert.

1. Ergänzen Sie Register A, Register B und gemeinsamen Speicher für alle Schritte. Welcher Endwert entsteht statt des gewünschten Wertes? (2 Punkte)
2. Reicht es, nur die beiden Stores einzeln zu schützen oder `volatile` hinzuzufügen? Begründen Sie den notwendigen Schutzumfang. (2 Punkte)

3. Bei einem Sensorpaar gehören `value` und `check` zum selben Update. Der Schreiber benutzt einen Mutex, der Leser nicht. Ist der gelesene Snapshot damit sicher konsistent? Beschreiben Sie eine korrekte Leseweise. (2 Punkte)

1. A liest 5, erhöht sein Register auf 6, Speicher bleibt 5. Beim Wechsel bleibt As 6 im gesicherten Kontext. B liest 5, erhöht sein Register auf 6 und schreibt 6. A schreibt anschließend seine 6. Der Speicher endet bei 6 statt 7.
2. Zwei einzeln geschützte Stores verhindern nicht, dass vorher beide denselben alten Wert lesen. Der gesamte zusammengehörige Lese-/Änder-/Schreibzugriff muss gegenüber dem anderen Teilnehmer unteilbar organisiert werden. `volatile` bietet diese Synchronisation nicht. Das gezeigte Maschinenmodell ist keine Empfehlung für tatsächlichen C-Mehrthreadcode mit konkurrierenden nichtatomaren Zugriffen: Ein Data Race kann dort undefiniertes Verhalten verursachen.
3. Der Leser kann zwischen den Schreiberzuweisungen landen und neuen `value` mit altem `check` erhalten. Er muss denselben Mutex erfolgreich erwerben, beide Felder in eine private Kopie übernehmen und freigeben. Längere private Berechnung folgt danach.

Aufgabe 2: Maskierungsbudget und Tokenzahl (8 Punkte)

1. Eine betroffene Interruptanforderung entsteht bei noch 8 µs verbleibender Maskierung. Der Eintritt benötigt danach angenommene 2 µs, die Reaktion weitere 3 µs. Weitere Kosten fehlen im Modell. Gilt $R \leq D$ für $D = 10\,\mu s$? Welcher maximale Maskierungsrest wäre unter denselben Annahmen zulässig? (3 Punkte)
2. Ein Semaphor wird mit Maximum 3 und Anfangszahl 3 angelegt. A, B und C erwerben je ein Token. D versucht Acquire mit Timeout 0. Danach gibt B frei, ohne dass ein anderer Thread wartet. D versucht erneut. Geben Sie die Zählerfolge und Ds beide Ergebnisse an. Ist mit dem Zähler die freie Pufferadresse bekannt? (3 Punkte)
3. Ein binärer Ereignissemaphor startet mit 0. Zwei Releases erfolgen, bevor jemand ein Token verbraucht. Wie viele Token sind anschließend verfügbar? Warum passt für „Ereignis noch nicht eingetreten“ der Startwert 0, für „Ressource anfangs verfügbar“ dagegen 1? (2 Punkte)

1. $R = (8 + 2 + 3)\,\mu s = 13\,\mu s > 10\,\mu s$. Die Modellfrist wird um 3 µs verfehlt. Der zulässige Maskierungsrest ist höchstens $(10 - 2 - 3)\,\mu s = 5\,\mu s$. Der Maskierungsumfang muss die tatsächlich konkurrierenden Interrupts umfassen; eine Threadmaskierung hält beispielsweise DMA nicht an.
2. Zählerfolge $3 \to 2 \to 1 \to 0$. Ds erster Versuch liefert einen Fehler ohne Blockieren. Bs Release erhöht auf 1; Ds **neuer** Versuch kann erwerben und reduziert auf 0. D war nach dem ersten Fehler kein registrierter Warter. Der Zähler gibt nur die Anzahl, nicht die Identität eines freien Puffers an.
3. Ein Token ist verfügbar. Das erste Release gelingt, das zweite scheitert am Maximum 1. Null lässt den Ereignis-Worker zunächst warten; eins stellt sofort eine Ressourcenerlaubnis bereit. Der Anfangswert beschreibt Bedeutung, nicht nur eine beliebige Konfiguration.

Aufgabe 3: Deadlock, Ordnung und Timeout (6 Punkte)

T1 hält Mutex A und wartet mit unbegrenzter Warteoption auf B. T2 hält B und wartet ebenso auf A. Es gibt genau eine exklusive Instanz je Ressource und keinen weiteren Abbruchpfad.

1. Zeichnen Sie den Ressourcenwartegraphen. Beschriften Sie die Bedeutung beider Pfeilrichtungen. Warum kann die betrachtete Gruppe nicht fortfahren? Müssen alle übrigen Threads und Interrupts stillstehen? (2 Punkte)
2. Erklären Sie die Coffman-Bedingung „No preemption“. Warum widerspricht sie einem präemptiven Prozessor-Scheduler nicht? Wie verhindert „immer A vor B“ den angegebenen Zyklus? (2 Punkte)
3. T1 bekommt beim Erwerb von B einen endlichen Timeout, hält aber schon A. Was muss sein Fehlerpfad tun? Warum garantieren Timeout und sofortiger Neuversuch allein keinen Fortschritt? (2 Punkte)

1. Mutex A → T1 → Mutex B → T2 → Mutex A. Ressource zum Thread bedeutet Besitz; Thread zur Ressource bedeutet Warten. Beide benötigen die vom jeweils anderen gehaltene Freigabe und bleiben Blocked. Andere Threads und Interrupts können trotzdem arbeiten.
2. „No preemption“ meint keinen zwangsweisen **Ressourcenentzug**, nicht fehlende Prozessorpräemption. Mit Ordnung A vor B darf T2 B nicht bereits halten, während es zuerst A fordert. Es wartet auf A ohne B; T1 kann B nehmen und beide freigeben.
3. Nach fehlgeschlagenem B-Erwerb nicht auf B zugreifen, gehaltenes A passend freigeben und teilweise Datenänderungen konsistent behandeln. Eine Wiederholungsregel muss festgelegt sein. Symmetrische sofortige Wiederholung kann fortgesetzte Konflikte ohne Nutzfortschritt erzeugen, einen Livelock. Prioritätsvererbung beseitigt den Ressourcenzyklus nicht.

Aufgabe 4: Derselbe Inversionsfall ohne und mit Vererbung (12 Punkte)

Ein Kern, feste Prioritäten $H > M > L$, einmalige Jobs, Wechsel- und API-Kosten null. API bedeutet Programmierschnittstelle.

Job	Freigabe	Prozessorbedarf
L	0 ms	nimmt Lock und benötigt insgesamt 5 ms bis zur Freigabe
H	1 ms	fordert denselben Lock, danach 1 ms eigene Arbeit
M	2 ms	20 ms Arbeit, greift nicht auf den Lock zu

Die relative Deadline von H ist $D_H = 10\,\text{ms}$. Der Anforderungsversuch von H hat im Modell keine eigene Dauer.

1. Zeichnen Sie die CPU-Belegung bis 26 ms **ohne** Vererbung. Bestimmen Sie Ls verbleibende Arbeit bei 1 und bei 2 ms. (3 Punkte)
2. Berechnen Sie Hs Lockwartezeit, Antwortzeit R_H und absolute Deadline. Ist die Frist eingehalten? (3 Punkte)
3. Zeichnen Sie dieselben Jobs **mit einem vererbenden Mutex**. Wann beginnt und endet Ls Anhebung? Bestimmen Sie Hs Warte- und Antwortzeit. (3 Punkte)
4. Vergleichen Sie die gesamte Prozessorarbeit. Welche Verzögerung wurde entfernt, welche nicht? Warum ist das kein Deadlock, und warum ist die endliche M-Arbeit von dauerhaftem Ready-Verhalten zu unterscheiden? (3 Punkte)

Ohne Vererbung:

Intervall in ms	CPU
$[0, 2)$	L, Hs dauerloser erfolgloser Versuch bei 1
$[2, 22)$	M
$[22, 25)$	L

Intervall in ms	CPU
$[25, 26)$	H

Bei 1 ms bleiben vier, bei 2 ms drei Millisekunden L-Arbeit. H wartet $25 - 1 = 24\,\text{ms}$ und hat $R_H = 26 - 1 = 25\,\text{ms}$. Seine absolute Deadline ist $d_H = 1 + 10 = 11\,\text{ms}$. Fertigstellung 26 ms liegt 15 ms danach.

Mit Vererbung: L arbeitet von 0 bis 5. Von 1 bis 5 hat es die hohe effektive Priorität, weil H am gehaltenen Mutex wartet. H läuft von 5 bis 6, M von 6 bis 26. H wartet $5 - 1 = 4\,\text{ms}$ und hat $R_H = 6 - 1 = 5\,\text{ms} \leq D_H$. Die Modellfrist wird gehalten.

In beiden Fällen beträgt die Gesamtarbeit $5 + 1 + 20 = 26\,\text{ms}$. Entfernt wurde M als zusätzliche Verdrängung des L-Halters vor H. Ls vier Millisekunden Restarbeit bleiben. L könnte bei Zuteilung fortfahren; es gibt keinen geschlossenen Ressourcenzyklus. Ein einmaliger M-Job endet nach 20 ms, ein dauerhaft bereites M könnte ohne Vererbung L unbegrenzt ausschließen. Bei mehreren Mutexen oder Gerätewartezeiten gilt dieses einfache Ein-Mutex-Ergebnis nicht pauschal.

Teil B: Kontrollierter Programmversuch

Aufgabe 5: CM4-Projekt und Anwendungsbaustein integrieren

Verwenden Sie den **funktionsfähigen Dual-Core-Grundstand aus Vorlesung 3**. In STM32CubeMX behalten Sie Board und Takte sowie FreeRTOS nur auf CM4 über CMSIS_V2 bei. Bauen und debuggen Sie die CMake-Ziele mit **VS Code und STM32 VS Code Extension**. Ein aktueller Generator muss die verwendete Kombination unterstützen; die einmalige Einrichtung wird nicht durch eine behauptete versionsunabhängige Menüfolge ersetzt.

1. Sichern Sie das bisherige Projekt. Halten Sie CubeMX-, CubeH7-, Compiler-, Extension-, Kernel- und Adapterversion fest. M7 bleibt beim dokumentierten Grundstart. Beide passenden Images werden geladen.

2. Verwenden Sie **nur drei Anwenderthreads** für diesen Versuch:

Rolle	CMSIS-Priorität	Stackattribut
L	`osPriorityNormal`	1024 Byte
M	`osPriorityAboveNormal`	1024 Byte
H	`osPriorityHigh`	1024 Byte

   Die Stackwerte sind Ausgangsgrößen, keine bewiesenen Minima. Entfernen oder deaktivieren Sie andere Anwenderlast, die alte HeavyMath-Schleife und kernelbezogene Tastercallbacks aus Vorlesung 3. Der Taster-IRQ bleibt für diesen Versuch gesperrt. Eine zusätzliche hochbereite Aufgabe würde den Vergleich ändern.

3. Prüfen Sie Präemption, 32-Bit-Kernelticks, 1000-Hz-Labortick, dynamische Objektanlage sowie Mutex-/Semaphorunterstützung in der erzeugten Konfiguration. Prüfen Sie die Prioritätsabbildung in `cmsis_os2.c`; die drei Rollen müssen tatsächlich verschieden und wie angegeben geordnet sein. Der Heap muss Threads, Gates, Testobjekt und übrigen Kernelbedarf tragen. Kontrollieren Sie erfolgreiche Anlage, Map-Datei und gegebenenfalls Stackreserve, statt eine beliebige Heapgröße als Nachweis einzusetzen.

4. Kopieren Sie `code/lab04/lab04_resources.c` und `lab04_resources.h` in einen eigenen CM4-Anwendungsordner. Fügen Sie Quelle und Include-Verzeichnis dem CM4-CMake-Ziel hinzu, über dessen vorhandene Benutzer-Erweiterungsstelle. Die Quelle benötigt die erzeugten Header `main.h` und `cmsis_os2.h`; sie ist kein selbstständiges Firmwareprojekt.

5. Im erhaltenen Benutzer-Include-Bereich der Threaddatei binden Sie `lab04_resources.h` ein. Ersetzen Sie die Körper der vorhandenen generierten Eintrittsfunktionen jeweils durch genau einen passenden Aufruf:

   ```
   /* In der jeweils passenden generierten Funktion, nicht dreimal in einer */
   Lab04_Low(argument);    /* L */
   Lab04_Medium(argument); /* M */
   Lab04_High(argument);   /* H */
   ```

   Verwenden Sie den tatsächlich generierten Argumentnamen. Jede Funktion kehrt nicht zurück. Es wird **keine zweite Definition** derselben Eintrittsfunktion erzeugt. Code steht in Funktions-USER-CODE-Bereichen, nicht in `Header_...`-Kommentaren.

6. Rufen Sie **nach dem vorhandenen `osKernelInitialize`, vor Anlage und Start der drei Threads** in einem erhaltenen Initialisierungsbereich einmalig auf:

   ```
   Lab04_Init(LAB04_SEMAPHORE);
   ```

   Der Kernel ist dabei `osKernelReady`. Der Baustein legt die Gates und den Testschutz an und prüft jedes Handle. Prüfen Sie auch alle generierten Threadhandles auf `NULL`. Der vorhandene `osKernelStart` bleibt einmalig; Rückkehr gehört in die Fehlerbehandlung. Eine Fehlerbehandlung darf nicht mit ungültigen Objekten weiterlaufen.

7. Lesen Sie die Startlogik und erklären Sie sie vor dem Flashen: H und M warten an **eigenen** Gates. L erwirbt den Testschutz und öffnet Hs Gate. H öffnet Ms Gate, bleibt zunächst höher als M und fordert dann den belegten Testschutz. Erst jetzt entscheidet die Vererbung, ob M oder L weiterarbeitet. Das zusätzliche dauerhaft leere Parking-Gate parkt beendete Einmal-Threads; `osDelay(osWaitForever)` wird nicht als Ersatz für diesen Objektwartevertrag benutzt.

8. Regenerieren und bauen Sie beide benötigten Ziele. Prüfen Sie, dass Ihre Benutzerbereiche und die CM4-CMake-Einbindung erhalten bleiben. Laden Sie die passenden Images und führen Sie den Semaphorversuch einmal aus.

Abgabe: Versionen, `.ioc`, Quellen, Build-/Debugkonfiguration und Ergebnis der Regenerierungsprüfung. Bei fehlender Hardware oder nicht unterstützter Werkzeugkombination dokumentieren Sie den konkreten Stand; Teil A bleibt vollständig lösbar.

Die Eintrittsfunktionen werden den drei Rollen eindeutig zugeordnet; die gezeigten drei Aufrufe sind Alternativen für drei verschiedene Funktionen. `Lab04_Init` wird einmal pro Neustart im initialisierten, noch nicht gestarteten Kernel aufgerufen.

Die beiden Start-Gates und das Parking-Gate beginnen leer. Das Ressourcentoken beginnt bei eins. L hält es bereits, wenn H freigegeben wird. H macht M vor seinem eigenen Blockieren bereit. Dadurch entsteht ohne Vererbung die Auswahl M vor L; beim vererbenden Mutex wird L durch Hs Warten angehoben. Das ist derselbe Mechanismus wie auf Papier, aber **nicht** automatisch dieselbe Freigabe bei 1/2 ms oder dieselbe Millisekundendauer.

Aufgabe 6: Derselbe Versuch mit Mutex

1. Ändern Sie ausschließlich den Modusaufruf zu `Lab04_Init(LAB04_MUTEX);`. Alle Iterationszahlen, Threadprioritäten, Gates, Takte und Optimierungen bleiben identisch.
2. Bauen, laden und **starten Sie kontrolliert neu**. Der Baustein ist ein Einmalversuch, kein Ablauf zum mehrfachen Initialisieren während eines laufenden Kernels.
3. Lesen Sie im Baustein die beiden vollständigen Objektpfade: Semaphor-New/Acquire/Release beziehungsweise Mutex-New/Acquire/Release. Ein Handle-Cast oder Umbenennen allein genügt nicht. Das Mutexattribut fordert `osMutexPrioInherit` an. Dessen Weglassen ist im geprüften öffentlichen FreeRTOS-Adapter kein verlässlicher Weg zum Abschalten der Vererbung; prüfen Sie Ihren lokalen Adapter.
4. Vergleichen Sie die Debugvariablen erst nach dem Lauf:

Beobachtung	Bedeutung
`lab04_log[LAB04_LOW]`	Ls eigene Zeit- und Zustandsmarken
`lab04_log[LAB04_MEDIUM]`	Ms erster Lauf und Arbeitsende
`lab04_log[LAB04_HIGH]`	Hs Anfrage, erfolgreicher Erwerb, Ende
`lab04_low_effective_priority`	Ls beobachtete wirksame Priorität nach H-Freigabe
`lab04_result[...]`	Ergebnis des identischen begrenzten Rechenpensums

   Jeder Thread schreibt nur seinen eigenen Log- und Ergebnisplatz. Hält ein Debugger den Prozessor an, verändert er das Zeitverhalten; solche Schritte sind keine ungestörte Zeitmessung. Ein Zustand `LAB04_LOCK_REQUEST` allein beweist keinen Kernelzustand Blocked: Kontrollieren Sie bei Bedarf die tatsächlichen Threadzustände beziehungsweise Traceereignisse.
5. Bestimmen Sie aus Hs Markern das beobachtete Lockintervall `acquired_tick - request_tick`. Vorzeichenlose Differenzbildung berücksichtigt einen einzelnen Zählerumlauf, sofern das Messintervall kleiner als eine volle Zählerperiode ist. Bei 1000 Hz ist ein Tick eine nominale Millisekunde. Gleiche Tickwerte lassen kurze Ereignisse nicht auflösen und beweisen keine Nullwartezeit.
6. Für ein **Marker-Antwortintervall** verwenden Sie Hs `done_tick` minus Ls `gate_release_request_tick`. Dieser Freigabemarker liegt **vor** dem Gate-Release-Aufruf. Das Intervall enthält daher dessen Kosten und ist nicht identisch mit einer isolierten Thread-CPU-Zeit. Legen Sie den Bezug explizit fest; setzen Sie nicht Hs Startmarker mit seiner Freigabe gleich.
7. Ls `release_request_tick` liegt vor dem Test-Release-Aufruf. Ls `done_tick` wird erst nach dessen Rückkehr geschrieben; da H oder M vorher laufen können, ist dieser Marker **nicht** die exakte Ressourcenfreigabezeit. Erfinden Sie daraus keine ungeprüfte Haltedauer.
8. Tragen Sie die tatsächlichen Werte ein. Lassen Sie ungemessene Felder offen. Falls Auflösung oder Ablauf zweifelhaft sind, verwenden Sie geeignete Traceereignisse oder getrennte geprüfte GPIO-Marker. GPIO ist ein allgemeiner Ein-/Ausgangspin; wählen Sie Belegung und Messmethode am Board. Live-`printf` bleibt aus diesem Timingvergleich entfernt.

Kontrollfeld	Semaphor	Mutex
L hält Testschutz vor H?		
H wartet tatsächlich am Testschutz?		
M-Gate vor Hs Warten geöffnet?		
Arbeitspensen und Build identisch?		
Ls beobachtete wirksame Priorität		
Hs Lockintervall, Einheit / Auflösung		
Hs Marker-Antwortintervall, Bezug		
Beobachtung / Messgrenze		

Diskussion: Welche Beobachtung spricht für entfernte M-Verdrängung im Haltefenster? Warum ersetzt ein bestandenes Ergebnis weder den kompletten Firmware-/Portnachweis noch eine allgemeine Deadlineanalyse?

Ohne Vererbung ist L nach seiner Gatefreigabe weiter normal priorisiert, M liegt darüber und kann vor Ls weiterem Ressourcenabschnitt rechnen. Mit dem vererbenden einzelnen Mutex sollte L im jeweiligen geprüften Adapter die hohe effektive Priorität von H erhalten und den Abschnitt vor M beenden können. H wartet weiterhin auf Ls verbleibende Arbeit.

Die Quelle enthält begrenzte 200000, 800000 und 1000 Iterationen für L, M und H. Daraus sind ohne Messung keine 5, 20 oder 1 ms ableitbar. Jeder Ergebnisplatz wird nur vom zugehörigen Thread bearbeitet; `volatile` erhält die Speicherzugriffe und ist kein allgemeines Lock. Die Gatephasen und Protokollierung erzeugen andere nichtnull Kosten als das idealisierte Papiermodell.

Erwartet wird ein dokumentierter Verlauf mit tatsächlicher Ressourcenabhängigkeit und unverändertem Arbeitspensum. Verglichen werden nicht nur zwei Endzahlen. Die Werte 24/4 ms und 25/5 ms gehören zur Papieraufgabe. Auch eine eindeutig verbesserte beobachtete H-Reaktion beweist nur den untersuchten Lauf und die dabei geprüften Bedingungen.

Zusatzfrage: Nachrichtenschutz und asynchroner Abschluss

Ein UART sendet 100 Nutzbytes bei 115200 Baud im Format 8N1. UART ist die asynchrone serielle Schnittstelle, 8N1 bedeutet acht Datenbits, kein Paritätsbit, ein Stoppbit; ein Startbit kommt hinzu.

1. Berechnen Sie die minimale reine Leitungszeit unter diesen Annahmen.
2. Zwei Threads sollen ganze Zeilen ausgeben. Warum werden privater Puffer und gemeinsamer UART-Mutex benötigt, und weshalb ist `printf` nicht automatisch durch diesen Mutex insgesamt threadsicher?
3. Bei IT-/DMA-Übertragung kehrt die Startfunktion vor dem tatsächlichen Abschluss zurück. Wann darf der Puffer wiederverwendet werden und wer gibt den CMSIS-Mutex frei? Macht Prioritätsvererbung die Leitung schneller?

$100 \cdot 10/115200\,\mathrm{s} \approx 8{,}68\,\mathrm{ms}$ reine Leitungszeit. Privater Puffer erhält die eigene Nachricht, der gemeinsame Vertrag serialisiert das Gerät. Bibliothekszustände und tatsächliches Retargeting der formatierten Ausgabe sind separat zu prüfen. Ein sichtbares Mischbild wird nicht als universelles Soll behauptet.

IT ist die interruptgesteuerte Übertragung, DMA direkter Hardware-Speicherzugriff. Der Puffer bleibt bis zu bestätigtem Abschluss oder sicherem Abbruch gültig und unverändert. Der Callback

darf zulässig signalisieren; der Besitzerthread gibt den Mutex frei. Ein Timeout allein bestätigt noch keine ruhende Hardware. Vererbung verändert die Taskzuteilung, nicht die Leitungsrate.

Abgabe und Quellen

Teil A: begründete Lösungen zu Aufgaben 1–4, insgesamt 32 Punkte. Teil B: vorbereiteter Grundstand, Integration, Regenerierungs-/Buildprüfung und beide beobachteten Varianten mit Grenzen der Auflösung. Unterscheiden Sie durchgeführt, fehlgeschlagen und nicht durchgeführt.

Quellen: Vorlesung 4, Arm CMSIS-RTOS2 zu Mutex und Semaphore, die **lokale** `cmsis_os2.c` und FreeRTOS-Konfiguration, ST UM2408/RM0399/AN5361 und Dokumentation der eingesetzten STM32 VS Code Extension. Der öffentliche Adapterstand ist keine automatisch bestätigte Firmwareversion Ihres Boards.